

Rust逆向之数据类型

与C/C++不同，Rust编译器是不允许数据类型互相隐式转换的。无论是无符号数和有符号数，亦或是字符类型或布尔类型与整型，还是u32和u8这种不同数据宽度。如果直接进行赋值，都会被编译器视为错误。

1.整型

在Rust中给整型赋值是不允许出现越界的情况，对于u8类型的整型，是不能直接赋值为-1。同样对于i8类型的整型，也不能直接赋值为255。否则，编译器会抛出错误。对于以下的代码：

```
fn test_num() {  
    let x: u8 = 0xFF;  
    let y: i8 = -1;  
}
```

所生成的汇编代码如下，可以看到这里是在编译器层面对变量进行了检查。其实在内存中，x和y都被赋值为0xFF(-1)。此外，和C/C++不同，先定义的变量会保存在栈的低地址，也就是变量的赋值会从栈顶到栈底赋值下去。另外，这里也可以看出，Rust在开辟栈空间的时候，是会根据变量所需要的内存大小，来开辟足够栈空间的。

```
.text:00000001400011A0 test_pro__test_num proc near  
.text:00000001400011A0                push    rax  
.text:00000001400011A1                mov     byte ptr [rsp+6], 0FFh ; x=0xFF  
.text:00000001400011A6                mov     byte ptr [rsp+7], 0FFh ; y=-1  
.text:00000001400011AB                pop     rax  
.text:00000001400011AC                retn  
.text:00000001400011AC test_pro__test_num endpest_num end
```

对于不同数据宽度的赋值，可以看以下代码：

```
fn test_num2() {  
    let x: u8 = 10;  
    let y: u32 = 10;  
}
```

对应的反汇编如下，同样的，这里是在编译器层面对数据的赋值进行了检查。在内存中，数据是一样的只是占用的宽度不同。

```

.text:00000001400011D0 test_pro__test_num2 proc near
.text:00000001400011D0             push     rax
.text:00000001400011D1             mov     byte ptr [rsp+3], 0Ah
.text:00000001400011D6             mov     dword ptr [rsp+4], 0Ah
.text:00000001400011DE             pop     rax
.text:00000001400011DF             retn
.text:00000001400011DF test_pro__test_num2 endp

```

在Rust中，定义的变量默认是不可变的，要想定义可变变量，需要加上mut关键字。对于以下的代码：

```

fn test_num3() {
    let x = 10;
    let mut y = 20;

    y = 30;
}

```

对应的反汇编如下，从汇编层面看，这里x和y并没有任何区别。所以mut关键字对于变量的可变性控制，依然是在编译器层面进行的。

```

.text:00000001400011E0 test_pro__test_num3 proc near
.text:00000001400011E0             push     rax
.text:00000001400011E1             mov     dword ptr [rsp], 0Ah ; x=10
.text:00000001400011E8             mov     dword ptr [rsp+4], 14h ; y=20
.text:00000001400011F0             mov     dword ptr [rsp+4], 1Eh ; y=30
.text:00000001400011F8             pop     rax
.text:00000001400011F9             retn
.text:00000001400011F9 test_pro__test_num3 endp

```

Rust还有一个特点，就是同一个变量名可以重复定义，对于以下代码：

```

fn test_num1() {
    let x = 10;
    let x = 20;
}

```

从汇编代码可以看出，第二次对x变量进行定义的时候，会将其保存在一块新的内存区域中。

```
.text:000000001400011B0 test_pro__test_num1 proc near
.text:000000001400011B0          push    rax
.text:000000001400011B1          mov     dword ptr [rsp], 0Ah ; x=10
.text:000000001400011B8          mov     dword ptr [rsp+4], 14h ; x=20
.text:000000001400011C0          pop     rax
.text:000000001400011C1          retn
.text:000000001400011C1 test_pro__test_num1 endp
```

Rust同样使用const和static，来定义常量和静态变量。对于以下代码：

```
fn test_num4() {
    const X: i32 = 10;
    static Y: i32 = 20;
    let x = X;
    let y = Y;
}
```

对于的反汇编如下，这里可以看出来。在Rust中，const关键字就类似于C/C++中的#define。也就是它会直接在编译阶段就把用到const关键字修饰的变量进行替换。

```
.text:000000001400011F0 test_pro__test_num4 proc near
.text:000000001400011F0          push    rax
.text:000000001400011F1          mov     dword ptr [rsp], 0Ah ; x=X
.text:000000001400011F8          mov     eax, cs:test_pro__test_num4__Y ; 获取Y变
量的值
.text:000000001400011FE          mov     [rsp+4], eax ; y=Y
.text:00000000140001202          pop     rax
.text:00000000140001203          retn
.text:00000000140001203 test_pro__test_num4 endp
```

而static关键字则是保存在.rdata数据段中，在编译阶段该变量就写好了。

```
.rdata:00000000140019350 ; int test_pro::test_num4::Y
.rdata:00000000140019350 test_pro__test_num4__Y dd 14h ; DATA XREF:
test_pro__test_num4+8↑r
```

除了不能直接给变量赋值超过变量范围的数值，像下面这样的代码，也会让编译器抛出error: this arithmetic operation will overflow的错误。

```
fn test_add() {
    let x: u8 = 254;
    let y: u8 = x + 10;
}
```

如果要通过编译，就需要把x的值修改为正确的范围内，以下是将x的值修改为245以后所对应的反汇编代码。可以看到，在对变量y进行赋值之前，会判断245+10是否产生了越界，如果没有越界，才会继续向下执行对y进行赋值。

```
.text:0000000140001180 test_pro__test_add proc near
.text:0000000140001180             sub     rsp, 28h
.text:0000000140001184             mov     byte ptr [rsp+26h], 245 ; x=245
.text:0000000140001189             mov     al, 245
.text:000000014000118B             add     al, 10 ; 将x+10的值保存到al
.text:000000014000118D             mov     [rsp+25h], al ; 保存x+10的值
.text:0000000140001191             setb    al ; CF为1，也就是add al,
10产生了进位，则设置al为1
.text:0000000140001194             test    al, 1
.text:0000000140001196             jnz     short loc_1400011A5 ; 如果al不为0则跳转
.text:0000000140001198             mov     al, [rsp+25h]
.text:000000014000119C             mov     [rsp+27h], al ; y=x+10
.text:00000001400011A0             add     rsp, 28h
.text:00000001400011A4             retn
```

如果发生了越界，则会跳转到以下代码进行报错：

```
.text:00000001400011A5 loc_1400011A5: ; CODE XREF:
test_pro__test_add+16↑j
.text:00000001400011A5             lea     rcx, aAttemptToAddWi ; "attempt to add
with overflow"
.text:00000001400011AC             lea     r8, off_140019360 ; "src\\main.rs"
.text:00000001400011B3             mov     edx, 1Ch
.text:00000001400011B8             call    _ZN4core9panicking5panic17h61d0277f5e1a7407E ;
core::panicking::panic::h61d0277f5e1a7407
```

2.字符型和布尔型

对于以下分别定义了字符型和整型的代码：

```
fn test_char() {
    let c: char = 'a';
    let x: u32 = 97;
}
```

对应的反汇编如下，这里可以看出字符型，在内存中就是其UTF-8对应的值。也因为Rust是UTF-8编码，所以字符型在内存中占了4个字节，和占用1个字节的C/C++是不同的。

```
.text:0000000140001210 test_pro__test_char proc near
.text:0000000140001210             push     rax
.text:0000000140001211             mov     dword ptr [rsp], 61h ; 'a'
.text:0000000140001218             mov     dword ptr [rsp+4], 61h ; 'a'
.text:0000000140001220             pop     rax
.text:0000000140001221             retn
.text:0000000140001221 test_pro__test_char endp
```

对于布尔型，考虑以下代码：

```
fn test_bool() {
    let b = true;
    let c = false;
    let x: i8 = 1;
}
```

对应反汇编可以看出，布尔型占用一个字节。true和false分别代表1和0，这和C/C++是一样的。

```
.text:0000000140001230 test_pro__test_bool proc near
.text:0000000140001230             push     rax
.text:0000000140001231             mov     byte ptr [rsp+5], 1
.text:0000000140001236             mov     byte ptr [rsp+6], 0
.text:000000014000123B             mov     byte ptr [rsp+7], 1
.text:0000000140001240             pop     rax
.text:0000000140001241             retn
.text:0000000140001241 test_pro__test_bool endp
```

3.枚举类型

对于枚举类型可以看如下代码：

```
fn test_enum() {
    enum Num {
        one,
        two,
        three,
        four,
        five,
    }

    let x = Num::one;
    let y = Num::two;
    let z = Num::five;
}
```

对应的反汇编如下，这里可以看出Rust中的枚举类型和C/C++一样，都是通过从0到N的不同数字，来代表不同的枚举值。不同的是，Rust中枚举类型的变量只占用1个字节。

```
.text:000000001400012B0 test_pro__test_enum proc near
.text:000000001400012B0          push     rax
.text:000000001400012B1          mov     byte ptr [rsp+5], 0 ; x=One
.text:000000001400012B6          mov     byte ptr [rsp+6], 1 ; y=two
.text:000000001400012BB          mov     byte ptr [rsp+7], 4 ; z=five
.text:000000001400012C0          pop     rax
.text:000000001400012C1          retn
.text:000000001400012C1 test_pro__test_enum endp
```

4. 数组

在Rust中数组和C/C++在内存中的布局是按顺序保存的，不过Rust中的数组是可以直接用=来赋值的。对于以下代码：

```
fn test_array() {
    let x = [1, 2, 3, 4, 5];
    let y = x;
    let z = 10;

    println!("{:?}", y);
}
```

对应反汇编如下，这里可以看出数组之间用=号赋值就是从相应内存区域中把值拿出来赋值到对应位置。然后，数组和数组之间先声明的数组会保存在栈顶。但是，对于i32的x，虽然它是后声明的，但是它会被保存在相比于先声明的数组更栈顶的地址。另外，这里的y数组，如果不对它进行使用，编译器会直接删掉相应的代码。

```

.text:0000000140001620 test_pro__test_array proc near
.text:0000000140001620             sub     rsp, 0B8h
.text:0000000140001627             mov     dword ptr [rsp+28h], 0Ah ; 为z赋值
.text:000000014000162F             mov     dword ptr [rsp+2Ch], 1 ; 为x数组赋值
.text:0000000140001637             mov     dword ptr [rsp+30h], 2
.text:000000014000163F             mov     dword ptr [rsp+34h], 3
.text:0000000140001647             mov     dword ptr [rsp+38h], 4
.text:000000014000164F             mov     dword ptr [rsp+3Ch], 5
.text:0000000140001657             mov     eax, [rsp+3Ch] ; 为y赋值
.text:000000014000165B             mov     [rsp+50h], eax
.text:000000014000165F             movups  xmm0, xmmword ptr [rsp+2Ch]
.text:0000000140001664             movaps  xmmword ptr [rsp+40h], xmm0
.text:0000000140001669             ; 省略println代码逻辑
.text:00000001400016E8             add     rsp, 0B8h
.text:00000001400016EF             retn
.text:00000001400016EF test_pro__test_array endp

```

在Rust中通过下标对数组访问的时候，编译器会进行检查。比如上面大小为5的x数组，如果在代码中写x[5]编译器会直接报错。包括以下形式的代码，如果编译器计算发现越界访问了，就会抛出error: this operation will panic at runtime的错误。

```

fn test_array() {
    let index = 4;
    let x = [1, 2, 3, 4, 5];
    let y = x[index + 1];
}

```

这里要通过编译就需要修改index，以下是将index修改为2以后的汇编代码。这里可以看到，在计算index+1的值之后，会将其与5进行比较。

```

.text:000000001400011C0 test_pro__test_array proc near
.text:000000001400011C0             sub     rsp, 48h
.text:000000001400011C4             mov     qword ptr [rsp+28h], 2 ; index=2
.text:000000001400011CD             mov     dword ptr [rsp+30h], 1 ; 为x数组赋值
.text:000000001400011D5             mov     dword ptr [rsp+34h], 2
.text:000000001400011DD             mov     dword ptr [rsp+38h], 3
.text:000000001400011E5             mov     dword ptr [rsp+3Ch], 4
.text:000000001400011ED             mov     dword ptr [rsp+40h], 5
.text:000000001400011F5             mov     eax, 2
.text:000000001400011FA             add     rax, 1
.text:000000001400011FE             mov     [rsp+20h], rax
.text:00000000140001203             setb   al
.text:00000000140001206             test   al, 1
.text:00000000140001208             jnz     short loc_14000121C ; 判断index + 1是否
产生了整型溢出
.text:0000000014000120A             mov     rax, [rsp+20h] ; rax=index+1
.text:0000000014000120F             cmp     rax, 5 ; 如果rax<5, cf会设置为1
.text:00000000140001213             setb   al ; 如果cf为1, 设置al为1
.text:00000000140001216             test   al, 1
.text:00000000140001218             jnz     short loc_140001234 ; 如果al不为1, 则跳
转
.text:0000000014000121A             jmp     short loc_140001246

```

如果index+1的值小于5，也就是没有发生数组越界。则会跳转到以下代码，对y进行赋值：

```

.text:00000000140001234 loc_140001234: ; CODE XREF:
test_pro__test_array+58↑j
.text:00000000140001234             mov     rax, [rsp+20h] ; rax=index+1
.text:00000000140001239             mov     eax, [rsp+rax*4+30h] ; eax = x的地址 +
(index + 1) * 4的地址
.text:0000000014000123D             mov     [rsp+44h], eax ; y=x[index+1]
.text:00000000140001241             add     rsp, 48h
.text:00000000140001245             retn

```

否则会跳转到以下代码进行报错：

```

.text:00000000140001246 loc_140001246: ; CODE XREF:
test_pro__test_array+5A↑j
.text:00000000140001246             mov     rcx, [rsp+48h+var_28]
.text:0000000014000124B             lea     r8, off_1400193B8 ; "src\\main.rs"
.text:00000000140001252             mov     edx, 5
.text:00000000140001257             call   _ZN4core9panicking18panic_bounds_check17h3d6d0774794afe02E ;
core::panicking::panic_bounds_check::h3d6d0774794afe02

```


5.字符串

字符串可以看以下代码：

```
fn test_str() {  
    let s1: &str = "Hello";  
    let s2 = s1;  
}
```

对应如下的反汇编，这里可以看出来。每个&str占0x10字节，其中前8个字节保存的是"Hello"字符串的地址aHellocalledOpt，后8个字节保存了该字符串的长度。&str的赋值不会从变量中获取地址，而是直接将aHellocalledOpt地址赋给对应的值。

```
.text:0000000140001160 test_pro__test_str proc near  
.text:0000000140001160             sub     rsp, 20h  
.text:0000000140001164             lea     rax, aHellocalledOpt ; "Hellocalled  
`Option::unwrap()` on a `No"...  
.text:000000014000116B             mov     [rsp], rax      ; 为s1赋值  
.text:000000014000116F             mov     qword ptr [rsp+8], 5  
.text:0000000140001178             lea     rax, aHellocalledOpt ; "Hellocalled  
`Option::unwrap()` on a `No"...  
.text:000000014000117F             mov     [rsp+10h], rax ; 为s2赋值  
.text:0000000140001184             mov     qword ptr [rsp+18h], 5  
.text:000000014000118D             add     rsp, 20h  
.text:0000000140001191             retn  
.text:0000000140001191 test_pro__test_str endp
```

这里的"Hello"字符串也是保存在.rdata数据段中，不过除了保存"Hello"字符串以外，后面还保存了一大串字符串。而这串字符串，看起来是用来检查这个&str的代码。但是具体这段代码在哪里执行，这里看不出来。

```
.rdata:0000000140019350 aHellocalledOpt db 'Hellocalled `Option::unwrap()` on a `None`  
value()',0  
.rdata:0000000140019350                                ; DATA XREF:  
test_pro__test_str+4↑o  
.rdata:0000000140019350                                ;  
test_pro__test_str+18↑o ...
```

6.元组

Rust中的元组允许将不同类型的值组成一个单一复合类型，以下是一个简单的实例：

```
fn test_tuple() {
    let t = (3, 'a', "Hello");

    let x = t.0;
    let y = t.1;
    let z = t.2;
}
```

对应的反汇编如下，可以看到，元组类型会按照顺序依次在内存中排序下去。使用的时候，也是从相应的内存地址将值取出直接使用。

```
.text:00000000140001160 test_pro__test_tuple proc near
.text:00000000140001160
.text:00000000140001160             sub     rsp, 30h
.text:00000000140001164             mov     dword ptr [rsp+4], 3 ; 为元组t赋值
.text:0000000014000116C             mov     dword ptr [rsp], 'a'
.text:00000000140001173             lea     rax, aHellocalledOpt ; "Hellocalled
`Option::unwrap()` on a `No"...
.text:0000000014000117A             mov     [rsp+8], rax
.text:0000000014000117F             mov     qword ptr [rsp+10h], 5
.text:00000000140001188             mov     eax, [rsp+4] ; eax=t.0
.text:0000000014000118C             mov     [rsp+18h], eax ; x=t.0
.text:00000000140001190             mov     eax, [rsp] ; eax=t.1
.text:00000000140001193             mov     [rsp+1Ch], eax ; y=t.1
.text:00000000140001197             mov     rcx, [rsp+8] ; rcx=t.2
.text:0000000014000119C             mov     rax, [rsp+10h]
.text:000000001400011A1             mov     [rsp+20h], rcx ; z=t.2
.text:000000001400011A6             mov     [rsp+28h], rax
.text:000000001400011AB             add     rsp, 30h
.text:000000001400011AF             retn
.text:000000001400011AF test_pro__test_tuple endp
```

7. 结构体

以下代码是Rust中，结构体的基本用法：

```

fn test_struct() {
    struct Test {
        x: i32,
        c: char,
    }

    let t = Test {
        x: 3,
        c: 'a',
    };

    let x = t.x;
    let c = t.c;
}

```

从下面的汇编代码可以看出，结构体的内存布局和上面说的元组是一样的：

```

.text:00000000140001160 test_pro__test_struct proc near          ; CODE XREF:
test_pro__main+4↑p
.text:00000000140001160
.text:00000000140001160          sub     rsp, 10h
.text:00000000140001164          mov     dword ptr [rsp+4], 3 ; 为结构体赋值
.text:0000000014000116C          mov     dword ptr [rsp], 'a'
.text:00000000140001173          mov     eax, [rsp+4]        ; eax=t.x
.text:00000000140001177          mov     [rsp+8], eax        ; x=t.x
.text:0000000014000117B          mov     eax, [rsp]          ; eax=t.c
.text:0000000014000117E          mov     [rsp+0Ch], eax      ; c=t.c
.text:00000000140001182          add     rsp, 10h
.text:00000000140001186          retn
.text:00000000140001186 test_pro__test_struct endp

```